SLIIT UNI
THE KNOWLEDGE UNIVERSITY

Sri Lanka Institute of Information Technology

Faculty of Computing

SE2020 - Web Technologies and Mobile

Ms. Dilani Lunugalage

BSc (Hons) in Information Technology

Lecture 03

Introduction to JavaScript

How JavaScript aligns with other Web Technologies

- HTML is for content
- CSS is for presentation
- JavaScript is for interactivity
  - validate user input in an HTML form before sending the data to a server;
  - build forms that respond to user input without accessing a server;
  - change the appearance of HTML documents and write data to browser Windows;
  - open and close new browser windows or frames;
  - build small but complete client side programs.

Things you can do with JavaScript - Some examples:

- Validate User Input
  - Checking the values in input fields
  - Checking a Password
  - Checking numeric values
- Forms that respond to user input without accessing a server
  - Forms that load a URL
  - Forms that calculate
- Write information into HTML documents using Javascript Statements
  - Programmed output via document.write() or [element selector].innerHTML
  - Adjust document colours
  - Change/rewrite images in a document

Things you can do with JavaScript - Some examples:

- Open and Close Browser windows and Frames
  - Create new Browser windows with documents or programmed output
- Build small but complete client side programs
  - Calculators
  - Games
- Control Robots
- Control drones (IOT)

The <script> </script> tag

```
<html>
<head>
  <title>The Script Tag 1</title>
  <script>
  document.write("<p>Script tags can be placed in the head of an HTML document.</p>");
  </script>
</head>
<body>
  <h1>The Script Tag: Example 1</h1>
  <p>
     Script tags can also be placed in the body of an HTML document.
  </p>
  <script>
document.write("<p>Script tags will be ignored by other browsers that do not understand them
  </script>
</body>
</html>
```

Script tags can be placed in the head of an HTML document.

The Script Tag: Example 1

Script tags can also be placed in the body of an HTML document.

Script tags will be ignored by other browsers that do not understand them.

JavaScript syntax basics

Comments

JavaScript supports single line and multi-line comments

```
// Single and multi line comments.
// this is an example of a single line comment.

/*
 * this is an example
 * of a
 * multi line
 * comment.
 */
```

White Space

```
const hello = "Hello";

const world     =      "World!";
```

It is a common practice to add whitespace to improve the code readability.

Here's an example with whitespace added for readability:

```
let foo = function() {
  for ( let i = 0; i < 10; i++ ) {
    alert( i );
  }
};

foo();
```

Reserved Keywords

- There are a handful of reserved words that can't be used when declaring user-defined variables and functions.
- Some of these reserved words are currently implemented, some are saved for future use, and others are reserved for historical reasons.
- A list of words and in-depth explanations for each can be found on the Mozilla Developer Network JavaScript Reference site.

Variables

Variables are containers for storing data. You can declare a variable in JavaScript:

- Using var
- Using let
- Using const

Declaring variables with var

- The var statement declares a function-scoped or globally-scoped variable, optionally initializing it to a value.
- In addition, variables declared with var are processed before any code is executed. This is called hoisting.
- The scope of a variable declared with var is its current execution context and closures thereof, which is either the enclosing function and functions declared within it, or, for variables declared outside any function, global. Duplicate variable declarations using var will not trigger an error, even in strict mode, and the variable will not lose its value, unless another assignment is performed.
- More on var

Declaring variables with let

- The let statement declares a block-scoped local variable, optionally initializing it to a value.
- let allows you to declare variables that are limited to the scope of a block statement, or expression on which it is used, unlike the var keyword, which declares a variable globally, or locally to an entire function regardless of block scope. The other difference between var and let is that the latter is initialized to a value only when a parser evaluates it
- More on let

Declaring variables with const

- const allows you to declare and initialize a read-only variable that will hold a constant value and something that we can never change.
- Const variables are block-scoped, much like variables declared using the let keyword.
- The value of a constant can't be changed through reassignment, and it can't be redeclared.
- However, if a variable declared with const is an object or array its properties or items can be updated or removed.
- An initializer for a constant is required. You must specify its value in the same statement in which it's declared.
- This declaration creates a constant whose scope can be either global or local to the block in which it is declared.
- More information on const

General Rule for declaring variables

- In general, use const for declaring objects or arrays, and variables that will not change.
- Use let to declare variables that will change through reassignment.
- Use var only if your code needs to run on in older browsers

Identifiers

- Identifiers are used to give variables and functions a unique name so they can subsequently be referred to by that name. The name of an identifier must follow a few rules:
- Cannot be a reserved word.
- Can only be composed of letters, numbers, dollar signs, and underscores.
- The first character cannot be a number.
- It's best practice to name identifiers in a way that will make sense to you and other developers later on.

```
// Valid identifier names.
const myAwesomeVariable = "a";
const myAwesomeVariable2 = "b";
const my_awesome_variable = "c";
const $my_AwesomeVariable = "d";
const _my_awesome_variable_$ = "e";
```

Types

- Types in JavaScript fall into two categories: primitives or objects. Primitive types include:
- String
- Number
- Boolean
- Null
- Undefined

Strings

- Strings are text wrapped in single or double quotation marks.It is best practice to consistently use one or the other. There may be times when the string contains quotation marks that collide with the ones used to create the string. In this case, either escape the characters using a \ backslash or use different quotes around the string.
- Strings can created with double or single quotes.

```
let a = "I am a string";
let b = 'So am I!';

alert( a );
alert( b );
```

Sometimes a string may contain quotation marks.

```
const statement1 = 'He said "JavaScript is awesome!"';
const statement2 = "He said \"JavaScript is awesome!\"";
```

```
const welcome = "Hello World";
```

Contd

```
let result = welcome.length; //result is the number 11

result = welcome.toUpperCase(); //result is HELLO WORLD

result = welcome.toLowerCase(); //result is hello world

result = welcome.charAt( 0 ); //result is the character H

result = welcome.substr( 6, 5 ); //(start,length) result is the string World

result = welcome.substring( 6, 11 ); //(start,end) result is the string World

result = welcome.indexOf( "World", 0 ); //return is the number 6.
//If the string is not found indexOf returns -1
```

```
Hello World
HELLO WORLD
hello world
H
Hello
6
```

Number

- Number types are any positive or negative numeric value. There is no distinction between integer and floating point values.

```
let num1 = 100;
let num2 = 100.10;
let num3 = 0.10;
```

- JavaScript is a loosely typed language. Programmers used to strongly typed languages such as C++ or Java find this difficult to get used to. For example in C/C++/Java the following statements, including the integer division,

```
int result;
result = 5/10;
```

- result would equal 0. In JavaScipt there are no explicit integers or floating-point numbers. "Unofficially," you will get one or the other back.

```
result = 5/10;
```

Boolean, Null and Undefined

- Boolean types are either true or false.

```
const okay = true;
const fail = false;
```

- Null and Undefined
  - Null types are a value that represent the absence of a value, similar to many other programming languages. Undefined types represent a state in which no value has been assigned at all. This type is created in two ways: by using the undefined keyword or by not defining a value at all.

```
// Achieve a null value
let foo = null;

// Two ways to achieve an undefined value.
var bar1 = undefined;
var bar2;
```

Objects

Everything else is in JavaScript is considered an Object. While there are numerous built-in objects, here we will cover:

- Date
- Array
- Function

Conditional Statements

In JavaScript we have the following conditional statements:

- **if statement** - use this statement to execute some code only if a specified condition is true
- **if...else statement** - use this statement to execute some code if the condition is true and another code if the condition is false
- **if...else if....else statement** - use this statement to select one of many blocks of code to be executed
- **switch statement** - use this statement to select one of many blocks of code to be executed

If Statement

Use the if statement to execute some code only if a specified condition is true.

Syntax

```
if (condition) {
  // code to be executed if condition is true
}
```

Example

```
if ( age < 16 ) {
  entry = "free";
}
```

If...else Statement

Use the if....else statement to execute some code if a condition is true and another code if the condition is not true.

Syntax

```
if ( condition ) {
  // code to be executed if condition is true
} else {
  // code to be executed if condition is not true
}
```

Example

```
if ( age < 16 ) {
  entry = "free";
} else {
  entry = "£10";
}
```

If...else if....else Statement

Use the if....else if...else statement to select one of several blocks of code to be executed.

Syntax

```
if ( condition1 ) {
  // code to be executed if condition1 is true
} else if ( condition2 ) {
  // code to be executed if condition2 is true
} else {
  // code to be executed if neither condition1 nor condition2 is true
}
```

Example

```
if ( age < 12 ) {
  entry = "free";
} else if ( age < 18 ) {
  entry = "£10";
} else {
  entry = "£20";
}
```

Question

Are these equivalent and why?

```
if ( age < 12 ) {
  entry = "free";
} else if ( age < 18 ) {
  entry = "£10";
} else {
  entry = "£20";
}
```

```
if ( age < 18 ) {
  entry = "£10" ;
} else if ( age < 12 ) {
  entry = "free";
} else {
  entry = "£20";
}
```

The JavaScript Switch Statement

Syntax

```
switch( n ) {
case 1:
  // execute code block 1
  break;
case 2:
  // execute code block 2
  break;
default:
  // code to be executed if n is different from case 1 and 2
}
```

The JavaScript Switch Statement

```javascript
var day = new Date().getDay();
switch ( day ) {
  case 0:
    message = "Today is Sunday";
    break;
  case 1:
    message = "Today is Monday";
    break;
  case 2:
    message = "Today is Tuesday";
    break;
  case 3:
    message = "Today is Wednesday";
    break;
  case 4:
    message = "Today is Thursday";
    break;
  case 5:
    message = "Today is Friday";
    break;
  case 6:
    message = "Today is Saturday";
    break;
}
```

Important Note: When the document.write() method is used to display content on the page, any prior content is removed. For this reason, document.write() is rarely used in production, and instead we prefer to select an element on the page and add content between its opening and closing tag (innerHTML). We will discuss this in more detail when we cover JavaScript and the DOM. In the meantime, have a look at the next interactive example: We add a div with the id="day" on the page, and in the JavaScript section, we select this div and make its innerHTML equal to x

Comparison Operators

Comparison operators are used in logical statements to determine equality or difference between variables or values.

```
x = 5;
```

Operator	Description	Comparing	Returns
==	is equal to	x == 8	false
		x == 5	true
===	is exactly equal to (value and type)	x === "8"	false
		x === 5	true
!=	is not equal to	x != 8	true
!==	is not equal to (neither value nor type)	x !== "5"	true
		x !== 5	false
>	is greater than	x > 8	false
<	is less than	x < 8	true
>=	is greater than or equal to	x >= 8	false
<=	is less than or equal to	x <= 8	true

Logical Operators

Logical operators are used to determine the logic between variables or values

```
x = 5;
y = 3;
```

Operator	Description	Example
&&	and	(x < 10 && y > 1) is true
&&	and	(x == 5 && y == 5) is false
||	or	(x == 5 || y == 5) is true
!	not	!(x == y) is true

JavaScript Loops

In JavaScript we have the following loops:

- **for** - loops through a block of code a number of times
- **for/in** - loops through the properties of an object
- **while** - loops through a block of code while a specified condition is true
- **do/while** - also loops through a block of code while a specified condition is true

The For Loop

The for loop is often the tool you will use when you want to create a loop. This will often be used if you want to repeat the same code a number of times, each time with a different value.

Syntax

```
for (initial statement; boolean expression; statement) {
  // the code block to be executed
}
```

- **initial statement** This statment is executed before looping begins.
- **boolean expression** This expression is evaluated to true or false. If true the loop is entered.
- **Statement** This statement is executed at the end of each loop.

Example

```
for ( let counter=0; counter < 10; counter++ ) {
   document.write( "<p>" );
   document.write( "Counter is: " + counter );
   document.write( "<p>" );
}
```

The While Loop

The while loop allows you to run a block of code while a condition is true. If the condition is not true at the start, the block of code inside the while loop will not be executed at all.

Syntax

```
while ( boolean expression ) {
   // the code block to be executed
}
```

- **boolean expression** This expression is evaluated to true or false. If true the loop is entered.
- **Statement(s)** Code block to be executed

Example

```
let counter = 0;
while( counter < 10 ) {
   document.write( "<p>" );
   document.write( "Counter is: " + counter );
   document.write( "</p>" );
   counter++;
}
```

The Do While Loop

- The do while loop allows you to run a block of code once, and to continue to run it while a condition is true.
- If the condition is not true at the start, the block of code inside curly braces will be executed once, as the condition check only takes place at the end of the execution of that block of code.
- This is particularly useful when asking for user input and validating it. You can ask the user to enter their input once, and to repeat as long as the value they have entered is invalid, see Example 2.

The Do While Loop

Syntax

```
do {
   // the code block to be executed
} while ( boolean expression );
```

- **boolean expression** This expression is evaluated to true or false. If true the loop is re-entered.
- **Statement(s)** Code block to be executed

Example 1

```
counter = 0;
do {
   document.write( "<p>" );
   document.write( "Counter is: " + counter );
   document.write( "</p>" );
   counter++;
} while ( counter < 10 );
```

The Do While Loop

```
let num1Valid = false;
let userNumber1;
do {
    userNumber1 = parseInt(prompt("Please choose a number betwee 1 and 59:"));
    if (!(Number.isInteger(userNumber1))) {
        alert("Error: The number you entered is not an integer");
    } else if ((userNumber1 < 1) || (userNumber1 > 59)) {
        alert("Error: The number must be between 1 and 59");
    } else {
        num1Valid = true;
    }
} while (num1Valid != true);
```

Note: In the do while, you will need to add a semicolon at the end of the while condition statement, unlike in the while condition.

The Break Statement

The **break** statement breaks the loop and continues executing the code after the loop (if any):

Example

```
let counter = 0;
while ( true ) {    //potentially infinite loop
   if ( counter > 5 ) {
   break;
   }
   counter++;
}
```

The Continue Statement

The **continue** statement breaks one iteration (in the loop), if a specified condition occurs, and continues with the next iteration in the loop.

Example

```
let counter = 0;
while ( counter < 6 ) {
   counter++;
   if ( counter % 2 == 0 ) {
     continue;
   }
   document.write( "Counter is now: " + counter );
}
```

JavaScript Dialog Boxes

alert();

Show a dialog box with a message

Example

```
alert( "Your Message Goes Here!" );
```

confirm();

Show a dialog box and return a boolean

Example

```
let answer = confirm( "Are you sure you want to close...." );
if ( answer == true ) {
  alert( "You said OK!" );
} else {
  alert( "You said Cancel!" );
}
```

prompt();

Show a dialog box and return a string

Example

```
let answer = prompt( "Enter something here:", "anything will do" );
alert( "You said " + answer );
```

Line breaks in dialog boxes

To display line breaks inside a popup box, use a backslash followed by the character n.

Example

```
alert( "Hello,\nHow are you?" );
```

Note: Even if your user enters a number in the prompt dialog box, this will be considered as a String. You will therefore need to convert it to a number using parseInt() or parseFloat() before being able to manipulate this with methods associated with numbers.

Review Questions

- Question 1 - Which HTML tags are used to hold JavaScript code?
- Question 2 - In which parts of the HTML document should the scripts go?
- Question 3 - Would it make sense to have a string holding HTML code as a parameter for the write() method?
- Question 4 - Would it make sense to have a string holding HTML code as a parameter for the alert() method?
- Question 5 - What are the 3 JavaScript dialog boxes and what are their differences?

JavaScript Objects

- In JavaScript, an object is data, with properties and methods.
- When you declare a JavaScript variable like this:

```
let message="Hello World!";
```

- You create a JavaScript String object.
- The String object has a built-in property called length. For the string above, length has the value 12.
- The String object also have several built-in methods.

Accessing Object Properties

Syntax

```
objectName.propertyName
```

Example

```
let message = "Hello world!";
let messageLength = message.length; //12
```

Accessing Object Methods

Syntax

```
objectName.methodName()
```

Example

```
let message = "Hello world!";
let newMessage = message.toUpperCase(); //HELLO WORLD!
```

JavaScript Arrays

An array is a special variable, which can hold more than one value at a time. There are various ways of creating an array:

1. Regular

```
const questions = [];
 questions[ 0 ] = "1 + 1";
 questions[ 1 ] = "2 x 3";
 questions[ 2 ] = "6 - 4";
```

2. Litteral

```
const questions = [ "1 + 1", "2 x 3", "6 - 4" ];
```

Array Properties

Property	Description
constructor	Returns the function that created the Array object's prototype
length	Sets or returns the number of elements in an array
prototype	Allows you to add properties and methods to an Array object

Array Methods

Method	Description

concat()	Joins two or more arrays, and returns a copy of the joined arrays
	Example
	`let fruits = ["Apple", "Banana", "Mango"];`
	`let vegetables = ["Carrot", "Potato"];`
	`let fruitsAndVeg = fruits.concat(vegetables);`
	`//fruitsAndVeg is equal to ["Apple", "Banana", "Mango", "Carrot", "Potato"]`

every()	Checks if every element in an array pass a test and returns a boolean value
	Example
	`function containsFiveLetters(currentValue) {`
	`  return currentValue.length == 5;`
	`}`
	
	`let fruit1 = ["Apple", "Banana", "Mango"];`
	`let fruit2 = ["Apple", "Mango"];`
	
	`console.log(fruit1.every(containsFiveLetters));`
	`//outputs: false`
	`console.log(fruit2.every(containsFiveLetters));`

Array Methods

fill()	Fills the elements in an array with a static value from an optional start index (default 0) to an optional end index (default array length)
	Example
	`let fruits = ["Apple", "Banana", "Mango", "Orange"];`
	
	`fruits.fill("Strawberry", 2, 4);`
	`// fills with "Strawberry" from position 2 until position 4`
	`console.log(fruits);`
	`//outputs ["Apple", "Banana", "Strawberry", "Strawberry"]`

filter()	Creates a new array with every element in an array that pass a test
	Example
	`function containsFiveLetters(currentValue) {`
	`  return currentValue.length == 5;`
	`}`
	
	`let fruits = ["Apple", "Banana", "Mango", "Orange", "Strawberry"];`
	`let result = fruits.filter(containsFiveLetters);`
	`//result is equal to ["Apple", "Mango"]`

Array Methods

find()	Returns the value of the first element in an array that passes a test
	Example
	`function containsSixLetters(currentValue) { return currentValue.length == 6; }`
	`let fruits = ["Apple", "Banana", "Mango", "Orange", "Strawberry"];`
	`let result = fruits.find(containsSixLetters);`
	`//result is equal to "Banana"`

findIndex()	Returns the index of the first element in an array that passes a test, otherwise returns -1
	Example
	`function containsSixLetters(currentValue) { return currentValue.length == 6; }`
	`let fruits = ["Apple", "Banana", "Mango", "Orange", "Strawberry"];`
	`let result = fruits.find(containsSixLetters);`
	`//result is equal to 1`

Array Methods

forEach()	Calls a function for each array element
	Example
	`let fruits = ["Apple", "Banana", "Mango"];`
	`fruits.forEach(function(element) {  console.log(element);});`
	
	`//outputs: "Apple"`
	`//outputs: "Banana"`
	`//outputs: "Mango"`

indexOf()	Searches the array for an element and returns its position
	Example
	`let fruits = ["Apple", "Banana", "Mango"];`
	`let pos = fruits.indexOf("Banana");`
	`// pos is equal to 1`

isArray()	Checks whether an object is an array
	Example
	`let fruits = ["Apple", "Banana", "Mango"];`
	`console.log(Array.isArray(fruits));`

Array Methods

join()	Joins all elements of an array into a string, and takes a separator as option
	Example
	`var fruits = ["Apple", "Banana", "Mango"];`
	`var myString = fruits.join(" * ");`
	`// myString is equal to: Apple * Bana  Mango`

lastIndexOf()	Search the array for an element, starting at the end, and returns its position
	Example
	`var fruits = ["Apple", "Banana", "Mango", "Banana", "Strawberry"];`
	`var pos = fruits.lastIndexOf("Banan`
	`// pos is equal to 3`

map()	Creates a new array with the result of calling a function for each array element
	Example
	`var array1 = [1, 4, 9, 16];`
	
	`function doubleValues(myVal) {`
	`  return( myVal * 2);`
	`}`

```
var map1 = array1.map(doubleValues);

console.log(map1);
// outputs: Array [2, 8, 18, 32]
```

Array Methods

pop()	Removes the last element of an array, and returns that element
	Example
	`var fruits = ["Apple", "Banana", "Mango"];`
	
	`console.log(fruits.pop());`
	`// outputs: "Mango"`
	
	`console.log(fruits);`
	`// outputs: Array ["Apple", "Banana"]`
	
	`fruits.pop();`
	
	`console.log(fruits);`
	`// outputs: Array ["Apple"]`

push()	Adds new elements to the end of an array, and returns the new length
	Example
	`var fruits = ["Apple", "Banana", "Mango"];`
	`fruits.push("Orange"); //["Apple", "Banana", "Mango", "Orange"]`

Array Methods

reduce()	Reduce the values of an array to a single value (going left-to-right) Example

```
let array1 = [1, 2, 3, 4];

function addUp(total, num) {
  return (total + num);
}

// 1 + 2 + 3 + 4
console.log(array1.reduce(addUp));
// outputs: 10

// 5 + 1 + 2 + 3 + 4
console.log(array1.reduce(addUp, 5));
// outputs: 15
```

Array Methods

reduceRight()	Reduce the values of an array to a single value (going right-to-left) Example

```
let array1 = [1, 2, 3, 4];

function takeAway(total, num) {
  return (total - num);
}

// 4 - 3 - 2 - 1
console.log(array1.reduceRight(takeAway));
// outputs: -2

// 5 - 4 - 3 - 2 - 1
console.log(array1.reduceRight(takeAway, 5));
// outputs: -5
```

reverse()	Reverses the order of the elements in an array. The original array is modified. Example

```
let fruits = ["Apple", "Banana", "Mango"];
fruits.reverse();
console.log( fruits );
//outputs: Array ["Mango", "Banana", "Apple"]
```

Array Methods

shift()	Removes the first element of an array, and returns that element Example

```
let fruits = ["Apple", "Banana", "Mango"];
let firstElement = fruits.shift();

console.log(fruits);
//outputs: Array ["Banana", "Mango"]

console.log(firstElement);
//outputs: "Apple"
```

slice()	Selects a part of an array, and returns the new array. The original array is not modified. Example

```
let fruits = ["Apple", "Banana", "Mango", "Banana", "Strawberry"];

console.log(fruits.slice(2));
//outputs: Array ["Mango", "Banana", "Strawberry"]

console.log(fruits.slice(2, 4));
//outputs: Array ["Mango", "Banana"]

console.log(fruits.slice(1, 5));
//outputs: Array ["Banana", "Mango", "Banana", "Strawberry"]
```

Array Methods

sort()	Sorts the elements of an array and returns the sorted array. The default sort order is ascending, built upon converting the elements into strings, then comparing their sequences of UTF-16 code units values, so should not be used for sorting number values. Example

```
var months = ['March', 'Jan', 'Feb', 'Dec'];
months.sort();
let ole.log(months);
//outputs: Array ["Dec", "Feb", "Jan", "March"]

var array1 = [1, 30, 4, 21, 100000];
array1.sort();
let ole.log(array1);
//outputs: Array [1, 100000, 21, 30, 4]
```

splice()	Adds/Removes elements from an array Example

```
var months = ['Jan', 'March', 'April', 'June'];
let hs.splice(1, 0, 'Feb');
//inserts at index 1
console.log(months);
//outputs: Array ['Jan', 'Feb', 'March', 'April', 'June']

months.splice(4, 1, 'May');
//replaces 1 element at index 4
console.log(months);
//outputs: Array ['Jan', 'Feb', 'March', 'April', 'May']
```

Array Methods

unshift()	Adds new elements to the beginning of an array, and returns the new length
	Example
	`var fruits = ["Apple", "Banana", "Mango"];`
	`let co .log(fruits.unshift("Strawberry", "Orange"));`
	`//outputs: 5`
	`console.log(fruits);`
	`//outputs: AArray ["Strawberry", "Orange", "Apple", "Banana", "Mango"]`

valueOf()	Returns the primitive value of an array
	Example
	`var fruits = ["Banana", "Orange", "Apple", "Mango"];`
	`let v = fruits.valueOf();`
	`let co e.log(v);`
	`//outputs: Array ["Banana", "Orange", "Apple", "Mango"]`

toString()	Converts an array to a string, and returns the result
	Example
	`let fruits = ["Apple", "Banana", "Mango"];`
	`let myString = fruits.toString();`
	`// myString is equal to: Apple, Banana, Mango`

Functions

- A function is a collection of JavaScript statements.
- These statements usually have a single purpose, such as performing a complex calculation or verifying the data entered into an HTML form.
- Functions can be passed copies of objects or variables to work with. This is done in the parameter list.
- JavaScript functions can be placed inside script tags anywhere in the document.
- However, they should be placed in the head of the document to guarantee they are loaded before being called from script statements in the body of the document.

Function

Syntax

```
function functionName( parameter_1, parameter_2, ..  parameter_n ) {
   statement1;
   statement2;
   .
   .
   .
   statementn;
}
```

Here is a function that finds that maximum value of two numbers.

```
function getMax( n1, n2 ) {
   if ( n1 < n2 ) {
      return n2;
   } else {
      return n1;
   }
```

Function Example

Here is an example of how this function might be used.

```
<html>
  <head>
    <script>
    function getMax( n1, n2 ) {
       if ( n1 < n2 ) {
          return n2;
       } else {
          return n1;
       }
    }
    </script>
  </head>
  <body>
   <script>
     document.write( "<p>" );
     document.write( "The largest number of 5 or 6 is: " + getMax( 5, 6 ) );
     document.write( "</p>" );
    </script>
  </body>
</html>
```

```
<html>
  <head>
    <script>
    function checkUserName( form ) {
       if ( form.reg.value.length != 8 ) {
          alert( "You have not entered an eight character string" );
          return false;
       } else {
          alert( "This is correct" );
          return true;
       }
    }
    </script>
  </head>
  <body>
   <p> Please enter your student username: <input type="text" name="reg" />
   <input type="button" onclick="checkUserName(form);" value="check validity"/>
  </body>
</html>
```

Global Functions

The JavaScript language comes with global properties and functions that can be used with all the built-in JavaScript objects:

decodeURI()	Decodes a URI
decodeURIComponent()	Decodes a URI component
encodeURI()	Encodes a URI
encodeURIComponent()	Encodes a URI component

eval()	Evaluates a string and executes it as if it was script code
	Example
	`var match4 = 140;`
	`var match3 = 30;`
	`var match = 3;`
	`match = "match" + match;`
	`alert("You win: £" + eval(match)); // alert("You win £30");`

isFinite()	Determines whether a value is a finite, legal number
isNaN()	Determines whether a value is not a number (NAN)
Number()	Converts an object's value to a number
parseFloat()	Parses a string and returns a floating point number
parseInt()	Parses a string and returns an integer
String()	Converts an object's value to a string

Thank You!